Authors

The work can be done in groups of up to 3 students. Please complete the following fields with your group number and list your names along with ISU ID numbers.

Technical Vision

1. Сухов Николай Михайлович, 368876
2. Иванов Никита Михайлович, 368225
3. Смирнов Алексей Владимирович, 409578

The task and guidelines were prepared by Andrei Zhdanov and Sergei Shavetov, ITMO University, 2025.

Practical Assignment No 3. Filtering and Edges Detection

Studying of the filtering images basic methods and edges detection.

Task 1. Noises types

Select an arbitrary image. Get images distorted by various noises with parameters other than the default values.

1.1 Preparation

First, we need to add some imports for OpenCV to work. Python implementation of OpenCV is in `cv2` library, but for convenience it will be imported as `cv`.

```
# Import OpenCV library both as cv and cv2
import cv2
import cv2 as cv
# Import NumPy library both as np and numpy
import numpy
import numpy as np
# Import skimage for noise generation functions
import skimage
# Import sys for general constants
import sys
```

Also we will import `ShowImages()` function from the first practical assignment to use it here. It is placed in `pa_utils`.

```
from pa_utils import ShowImages
```

1.2 Read and display an image

Now let's open our image which we will use during the current task. We will open it in BGR and convert to grayscale. We will use previously introduced functions for image display.

```
# Read an image from file in BGR
fn = "images/lena_color.png"
I = cv.imread(fn, cv.IMREAD_COLOR)
if not isinstance(I, np.ndarray) or I.data == None:
  print("Error reading file \"{}\"".format(fn))
else:
  # Convert BGR to grayscale
  I = cv.cvtColor(I, cv.COLOR_BGR2GRAY)
  # Display it
  ShowImages([("Source image", I)])
```

1.3 Noises

Digital images obtained by various optoelectronic devices can contain various distortions caused by various kinds of interference, which are commonly called *noise*. The noise in the image makes it difficult to process it automatically. The noises have a different nature, and for its successful suppression it is necessary to determine an adequate mathematical model. Let's consider the most common noise models. When using OpenCV noise have to be added manually, however the `skimage` Python toolbox provides `skimage.util.random_noise()` function which allows adding noise. We will try both ways.

1.3.1 Impulse noise

With impulse noise, the signal is distorted by spikes with very large negative or positive values of short duration and can arise, for example, due to decoding errors. This noise results in white "*salt*" or black "*pepper*" dots in the image, which is why it is often called *dot* noise. To describe it, one should take into account the fact that the appearance of a noise spike in each pixel $I(x, y)$ does not depend on the quality of the original image or on the presence of noise at other points and has the probability of p, and the value of the pixel intensity $I(x, y)$ will be changed to value $d \in [0, 255]$:

$$I_{noisy}(x, y) = \begin{Bmatrix} d & \text{with probability } p \\ I(x, y) & \text{with probability } (1 - p) \end{Bmatrix}$$

where $I(x, y)$ is the pixel intensity of the original image, I_{noisy} is the noisy image, and p is the nose ratio. If $d = 0$ then the noise "*pepper*" type, if $d = 255$ then noise is "*salt*".

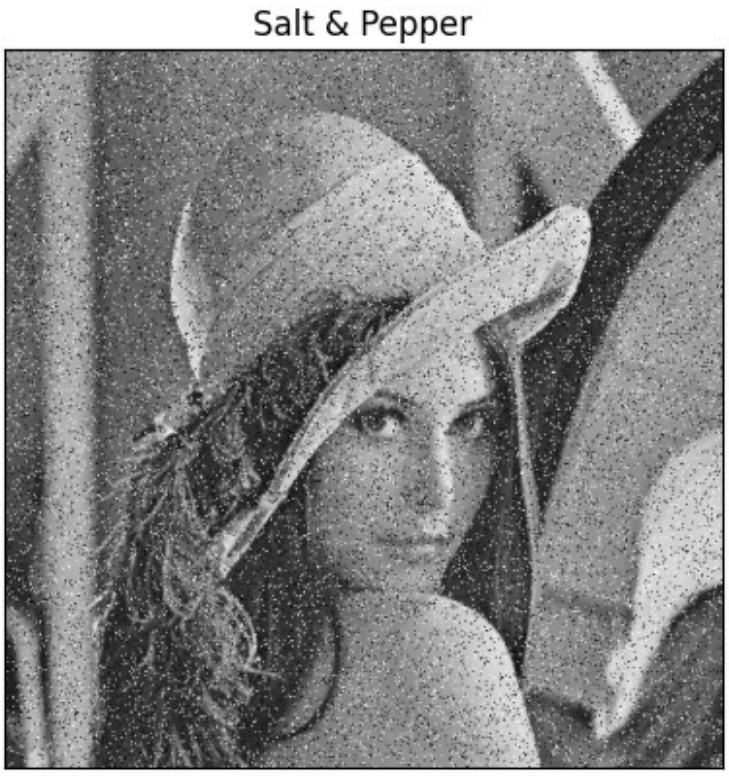
Let's implement the impulse noise. As for the *salt* type noise we have to add the maximum value, we will consider that only floating point images with 1.0 maximum value, or `uint8` ones with 255 maximum value are supported. The noise is controlled by two values: the noise probability (`p`) and salt versus pepper ratio (`s_vs_p`).

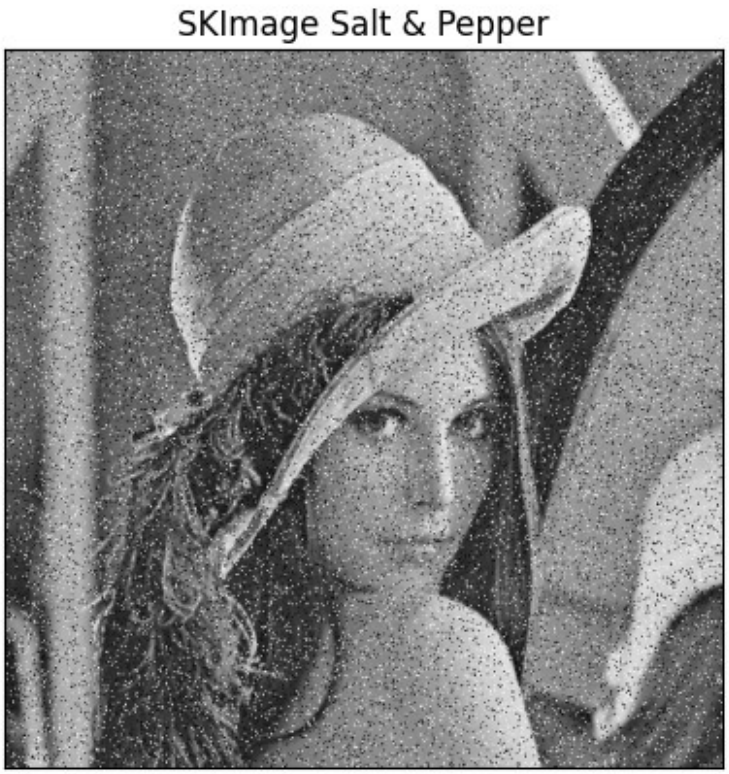
```
## Add the salt and pepper noise to an image
## @param[in] image An image
## @param[in] p Probability of noise
## @param[in] s_vs_p Salt vs pepper distribution
## @return An image with noise
def SaltAndPepper(I, p = 0.05, s_vs_p = 0.5):
  # Select the salt value
  if I.dtype == np.uint8:
    salt = 255
  elif I.dtype == np.float32:
    salt = 1.0
  else:
    print("Unsupported image type")
    return None

  # Create an image
  Inoise = np.copy(I)
  # Get random value for each image poxel
  rng = np.random.default_rng()
  vals = rng.random(Inoise.shape)
  # Add salt
  Inoise[vals < p * s_vs_p] = salt
  # Add pepper
  Inoise[np.logical_and(vals >= p * s_vs_p, vals < p)] = 0

  return Inoise
```

Alternatively, the `skimage.util.random_noise(I, 's&p')` function can be used. Its optional `amount` parameter can be used to modify the "*salt*" and "*pepper*" probabilities. The next optional `salt_vs_pepper` can be used to set the relative probabilities of "*salt*" and "*pepper*" noises. Only "*salt*" noise can be generated by `skimage.util.random_noise(I, 'salt')` function, while only "*pepper*" can be generated by `skimage.util.random_noise(I, 'pepper')` function. It should also be noted that these methods converts an image to a floating point value image, so additional conversion is required.

Let's run both this methods and compare results.

```
# Own implementation
Isnp = SaltAndPepper(I, 0.1, 0.5)
# Skimage
Isnp2 = (skimage.util.random_noise(I, "s&p", amount = 0.1,
salt_vs_pepper = 0.5) * 255).astype(np.uint8)
# Show both
ShowImages([("Salt & Pepper", Isnp),
            ("SKImage Salt & Pepper", Isnp2)])
```





We may also approximately calculate the resulting *salt* and *pepper* ratios with simple math transformations. Let's do it.

```
diff = I.astype(np.int32) - Isnp
print("Own implementation salt ratio is {:.2f}%, pepper ratio is
{:.2f}%".format(
      100 * np.count_nonzero(diff < 0) / I.shape[0] / I.shape[1],
      100 * np.count_nonzero(diff > 0) / I.shape[0] / I.shape[1]))
diff2 = I.astype(np.int32) - Isnp2
# As skimage work in floating point values with conversion,
# so we have to take this into an account and add some epsilon value
print("Skimage implementation salt ratio is {:.2f}%, pepper ratio is
{:.2f}%".format(
      100 * np.count_nonzero(diff2 < -1) / I.shape[0] / I.shape[1],
      100 * np.count_nonzero(diff2 > 1) / I.shape[0] / I.shape[1]))
```

```
Own implementation salt ratio is 4.97%, pepper ratio is 5.01%
Skimage implementation salt ratio is 5.03%, pepper ratio is 4.96%
```

1.3.2 Additive noise

Additive noise is described by the following expression:

$$I_{noisy}(x,y)=I(x,y)+\eta(x,y)$$

where I_{noisy} is the noisy image, I is the original image, and η is the signal-independent additive noise with Gaussian or any other probability density function.

We will see an example of an additive noise a bit later in a Gaussian noise section.

1.3.3 Multiplicative Noise

Multiplicative noise is described by the following expression:

$$I_{noisy}(x,y) = I(x,y) \cdot \eta(x,y)$$

where I_{noisy} is the noisy image, I is the original image, and η is the signal independent multiplicative noise that multiplies the recorded signal. Examples include film grain, ultrasound images, etc. A special case of multiplicative noise is *speckle* noise. This noise appears in images captured by coherent imaging devices such as medical scanners or radars. In such images, one can clearly observe light spots, speckles, which are separated by dark areas of the image.

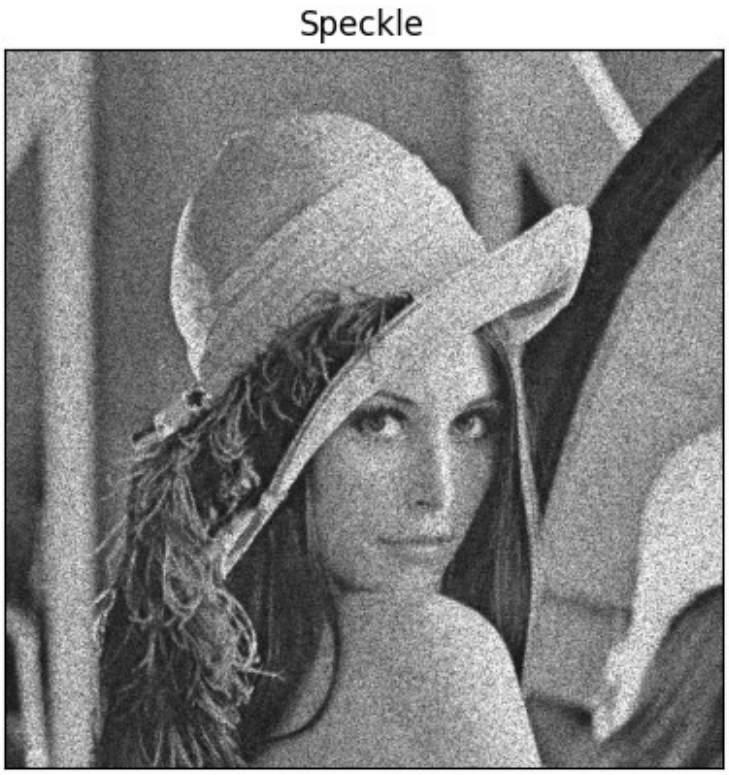
Let's implement the multiplicative noise with normal distribution which is called the *speckle* noise.

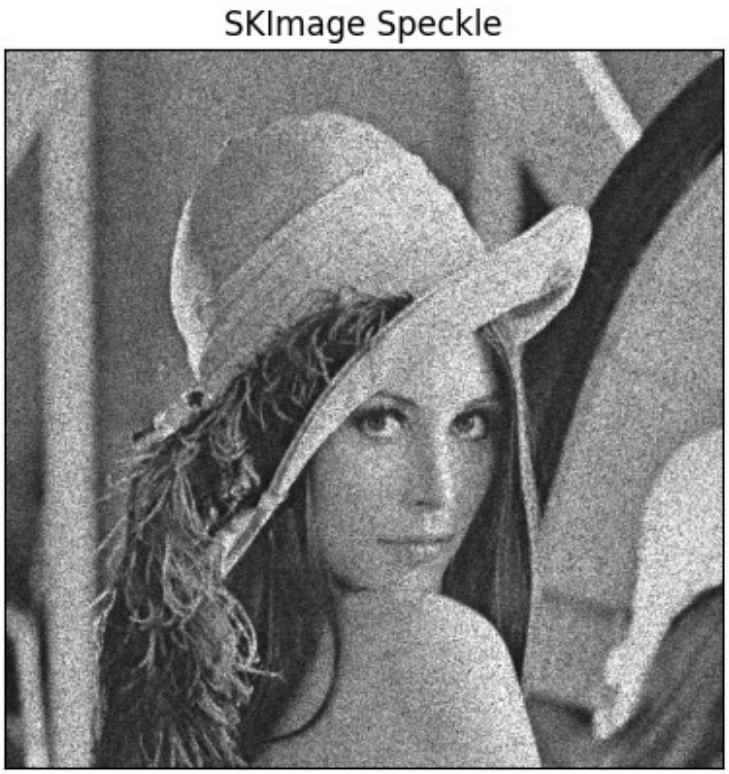
```
## Add the speckle noise to an image
## @param[in] image An image
## @param[in] var Random number variance
## @return An image with noise
def Speckle(I, var = 0.05):
  if I.dtype != np.uint8 and I.dtype != np.float32:
      print("Unsupported image values type.")
      return None

  # Generate a random number with normal distribution for each image
pixel
  rng = np.random.default_rng()
  gauss = rng.normal(0, var ** 0.5, I.shape)

  # Add a multiple of this distribution to our image
  # The image values type should be taken into an account here
  # as we should work with floating point values
  if I.dtype == np.uint8:
    out = (I.astype(np.float32) * (gauss + 1)).clip(0,
255).astype(np.uint8)
  else:
    out = I * (gauss + 1)

  return out
```

This noise can also be applied to an image by using `skimage.util.random_noise(I, 'speckle')`. Optional `mean` and `var` parameters defines the mean and variance for the normal distribution used to generate noise.

Let's run both this methods and compare results.

```
# Own implementation
Isp = Speckle(I, 0.05)
# Skimage
Isp2 = (skimage.util.random_noise(I, "speckle", mean = 0, var = 0.05)
* 255).astype(np.uint8)
# Show both
ShowImages([("Speckle", Isp),
            ("SKImage Speckle", Isp2)])
```





1.3.4 Gaussian (Normal) Noise

Gaussian noise in the additive noise on the image that can occur due to a lack of scene illumination, high temperature, etc. The noise model is widely used in low-pass filtering of images. The probability density distribution function $p(z)$ of the random variable z is described by the following expression:

$$p(z)=\frac{1}{\sigma\sqrt{2\pi}}e^{\frac{-(z-\mu)^2}{2\sigma^2}}$$

where z is the the intensity of the image (for example, for a grayscale image $z \in [0,255)$), μ is the mean (mathematical expectation) of a random variable z, σ is the standard deviation, variance σ^2 determines the power of the introduced noise. Approximately 67 of the random variable z values is concentrated in the range $[(\mu-\sigma),(\mu+\sigma))$ and about 96 in the range $[(\mu-2\sigma),(\mu+2\sigma))$.

As we know that this is an additive noise and it uses the normal distribution, we can easily implement it.

```
## Add the Gaussian noise to an image
## @param[in] image An image
## @param[in] mean Mean value
## @param[in] var Variance
## @return An image with noise
def Gaussian(I, mean = 0, var = 0.01):
```

```
  if I.dtype != np.uint8 and I.dtype != np.float32:
    print("Unsupported image values type.")
    return None

  rng = np.random.default_rng()
  gauss = rng.normal(mean, var ** 0.5, I.shape)

  if I.dtype == np.uint8:
    out = (I.astype(np.float32) + gauss * 255).clip(0,
255).astype(np.uint8)
  else:
    out = (I + gauss).astype(np.float32)

  return out
```

It can also be applied to an image by using `skimage.util.random_noise(I, 'gaussian')` function. Optional `mean` and `var` parameters defines the mean and variance for the normal distribution used to generate noise. The additional `skimage.util.random_noise(I, 'localvar')` function allows to specify the local random variance for each pixel of the image passed by an extra `local_vars` function parameter.

Let's run both this methods and compare results.

```
# Own implementation
Igauss = Gaussian(I, 0, 0.05)
# Skimage
Igauss2 = (skimage.util.random_noise(I, "gaussian", mean = 0, var =
0.05) * 255).astype(np.uint8)
# Show both
ShowImages([("Gaussian", Igauss),
            ("SKImage Gaussian", Igauss2)])
```

Gaussian

SKImage Gaussian

1.3.5 Quantization Noise

Depends on the selected quantization step and on the signal. Quantization noise can lead, for example, to the appearance of false contours around objects or to remove low-contrast details in the image. Such noise is not eliminated. Quantization noise can be approximately described by the Poisson distribution.

Since we know the distribution type, we can implement this noise.

```
## Add the poison noise to an image
## @param[in] image An image
## @return An image with noise
def Poisson(I):
  rng = np.random.default_rng()
  # We have to calculate the number of unique values then add
  # the poisson distribution noise basing on this number to
  # simulate the quantization noise
  if I.dtype == np.uint8:
    Ifloat = I.astype(np.float32) / 255
    vals = len(np.unique(Ifloat))
    vals = 2 ** np.ceil(np.log2(vals))
    out = (255 * (rng.poisson(Ifloat * vals) / float(vals)).clip(0,
1)).astype(np.uint8)
  elif I.dtype == np.float32:
    vals = len(np.unique(I))
    vals = 2 ** np.ceil(np.log2(vals))
    out = rng.poisson(I * vals) / float(vals)
  else:
    print("Unsupported image values type.")
    return None

  return out
```

It can also be applied by using `skimage.util.random_noise(I, 'poisson')` function.

Let's run both this methods and compare results.

```
# Own implementation
Ipoisson = Poisson(I)
# Skimage
Ipoisson2 = (skimage.util.random_noise(I, "poisson") *
255).astype(np.uint8)
# Show both
ShowImages([("Poisson", Ipoisson),
            ("SKImage Poisson", Ipoisson2)])
```

Poisson

SKImage Poisson

1.3.6 Noise summary

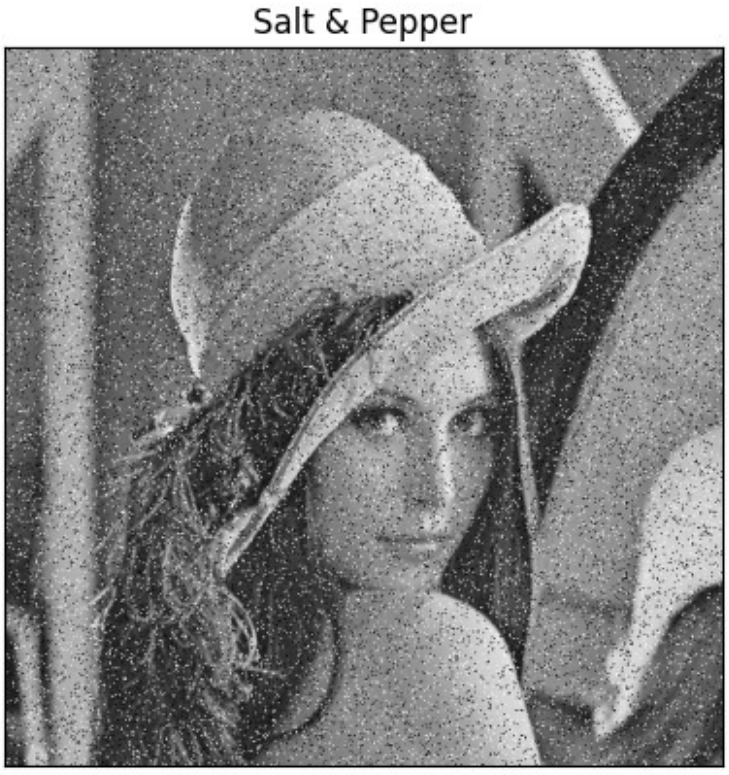
So, we learned how to add different type of noise. Let's write a copy of MATLAB's `imnoise()` function.

```
## Add noise to an image
## @param[in] image An image
## @param[in] noise_type Noise type to add
## @param[in] param1 First noise-dependent parameter
## @param[in] param2 Second noise-dependent parameter
## @return An image with noise
## @note For "salt & pepper" type, param1 is the probability, param2
is the salt vs pepper ratio.
## @note For "speckle" type, param1 is variance.
## @note For "gaussian" type, param1 is mean, param2 is variance.
def imnoise(I, noise_type, param1 = None, param2 = None):
  # Salt & pepper (param1 - probability, param2 - salt vs pepper)
  if noise_type == "salt & pepper":
    if param1 != None:
      d = param1
    else:
      d = 0.05
    if param2 != None:
      s_vs_p = param2
    else:
      s_vs_p = 0.5

    return SaltAndPepper(I, d, s_vs_p)

  # Multiplicative Noise (param1 - variance)
  if noise_type =="speckle": # Variance of multiplicative noise,
specified as a numeric scalar
    if param1 != None:
      var = param1
    else:
      var = 0.05

    return Speckle(I, var)

  # Gaussian Noise (param1 - mean, param2 - variance)
  if noise_type == "gaussian":
    if param1 != None:
      mean = param1
    else:
      mean = 0
    if param2 != None:
      var = param2
    else:
      var = 0.01
```

```
    return Gaussian(I, mean, var)

  # Quantization Noise
  if noise_type == "poisson":
    return Poisson(I)

  return None
```

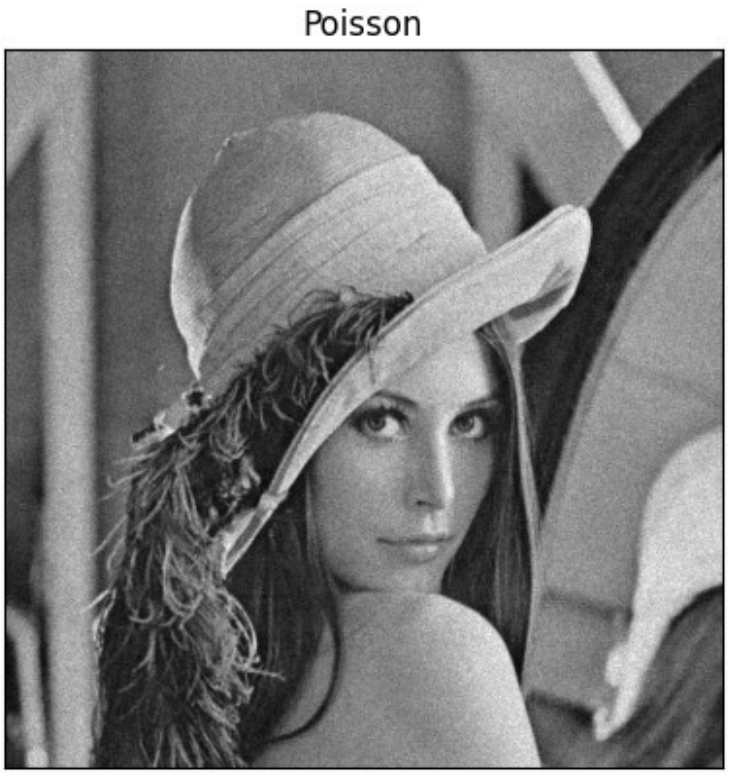
Also, let's make a more convenient array of images to test different noise filters.

```
Itmp = I # cv.resize(I, (256, 256))
Inoise = [("Salt & Pepper", SaltAndPepper(Itmp, 0.1)),
          ("Speckle", Speckle(Itmp)),
          ("Gaussian", Gaussian(Itmp)),
          ("Poisson", Poisson(Itmp))]
ShowImages(Inoise, 0)
```



Speckle

Gaussian



1.3.6.1 Self-work

Self-work

Take some arbitrary image and apply different noises to it. Use $6\,\%$ percent probability for the impulse noise with $1:1$ distribution of salt versus pepper ($50\,\%$ percent). Show the results and store them in an array to test filters.

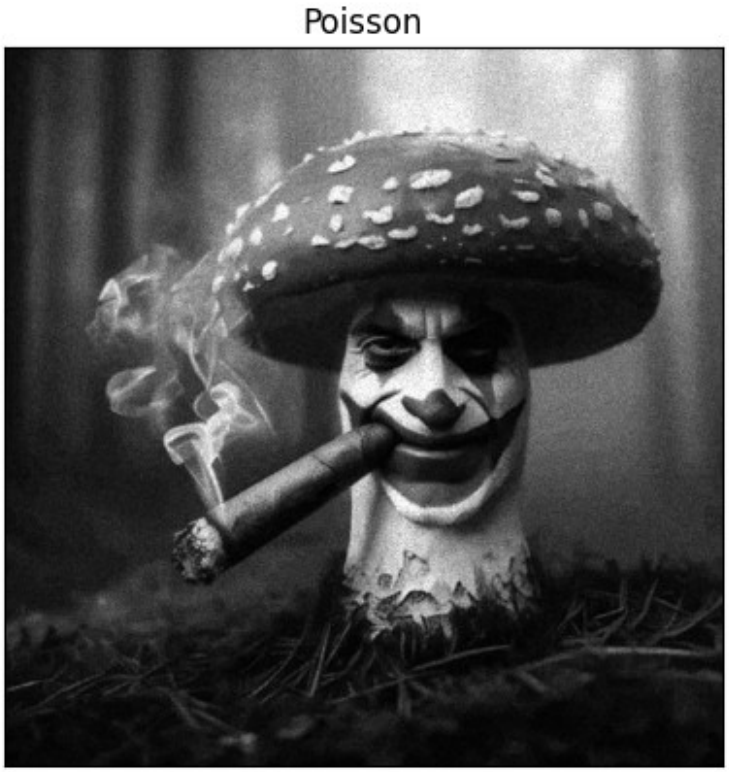
```
I_jok = cv.imread("images/jokshroom.jpg", cv.IMREAD_COLOR)
I_jok = cv.cvtColor(I_jok, cv.COLOR_BGR2GRAY)
I_jok = I_jok.copy()

Inoise = [
    ("Salt & Pepper (p=0.06, 1:1)", SaltAndPepper(I_jok, 0.06, 0.5)),
    ("Speckle", Speckle(I_jok)),
    ("Gaussian", Gaussian(I_jok)),
    ("Poisson", Poisson(I_jok)),
]

ShowImages([("Source image", I_jok)] + Inoise, 0)
```

Source image

Salt & Pepper (p=0.06, 1:1)

Speckle

Gaussian



Task 2. Low-pass filtering

Process the distorted images obtained in the previous paragraph with considered low-pass filters and a counterharmonic mean filter with different values of the parameter Q.

2.1 Sliding window filters

Let's consider the main methods of image filtering. It the neighboring pixels values in a certain neighborhood are taken into account to calculate th intensity value of the each pixel, then such a transformation is called \text{local}, and the neighborhood is called a *window*. The "window" is called *mask*, *filter*, *filter kernel*, and matrix coefficients are called *coefficients*. The center of the mask is aligned with the analyzed pixel, and the mask coefficients are multiplied by the intensities of the pixels covered by the mask. Typically, the mask has a square shape of 3×3, $5\ times 5 $, etc. Filtering an $M \times N$ image I using a mask of $m \times n$ size is described by the formula:

$$I_{filtered}(x, y) = \sum_{s} \sum_{t} w(s,t) I(x+s, y+t)$$

where s and t are the coordinates of the mask elements relative to its center (in the center $s = t = 0$). Such transformations are called *linear*. After calculating the new value of the pixel intensity $I_{filtered}(x, y)$ window w, n which the filter mask is described, it is shifted and the intensity of the next pixel is calculated. Therefore, such a transformation is called *sliding window filtering*.

When using OpenCV image filtering can be performed by calling the `dst = cv2.filter2D(src, ddest, kernel)` function. Here `src` is an input image matrix, `dst` is

an output image matrix, `kernel` is filtering kernel that should be specified manually. Filtering mask can be created manually by creating new numpy matrix, for example:

```
mask = np.float64([[1, 1, 1], [1, 1, 1], [1, 1, 1]])
```

2.2 Low-pass filter kernels

Low-pass spatial filters attenuate the high-pass components (areas with large intensity changes) and leave the low-pass components of the image unchanged. Used to reduce noise and remove high-pass components, which improves the accuracy of the study of the content of low-pass components. As a result of applying low-pass filters, we get a smoothed or blurred image. The main distinguishing features are:

- non-negative mask coefficients;
- the sum of all coefficients is equal to one.

Examples of low-pass filter kernels:

$$w_1=\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}, w_2=\frac{1}{10}\begin{bmatrix}1 & 1 & 1\\ 1 & 2 & 1\\ 1 & 1 & 1\end{bmatrix}.$$

Let's try running the low-pass filter with these kernels for all our four noise types. First we implement kernel w_2.

```
# Define a kernel
w1 = np.ones((3, 3)) / 9
# Create a storage for the images filtered with a low-pass filter
Iw1 = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
  Iw1.append((img[0], cv.filter2D(img[1], -1, w1)))
# Show filtered images
ShowImages(Iw1, 0)
```

Salt & Pepper (p=0.06, 1:1)

Speckle

Gaussian

Poisson

2.2.1 Self-work

Self-work

Run the low-pass 2D filter with kernel w_2 for all noise types.

```
# Low-pass kernel w2
w2 = np.array([[1, 1, 1],
               [1, 2, 1],
               [1, 1, 1]], dtype=np.float32) / 10.0

# Apply filter to all noisy images
Iw2 = []
for name, img in Inoise:
  Iw2.append((name, cv.filter2D(img, -1, w2)))

ShowImages(Iw2, 0)
```

Speckle

Gaussian

2.3 Arithmetic mean filter

This filter averages the pixel intensity value over the neighborhood using a mask with the same coefficients, for example, for a mask with the size 3×3 the coefficients are $1/9$, if 5×5 --- $1/25$. Thanks to this normalization, the value of the filtering result will be reduced to the original image intensities range. Graphically, the two-dimensional function describing the filter mask looks like a parallelepiped, so the name is used in English literature *box*-filter. Arithmetic averaging is achieved using the following formula:

$$I_{filtered}(x, y) = \frac{1}{m \cdot n} \sum_{i=0}^{m} \sum_{j=0}^{n} I(i, j)$$

where $I_{filtered}(x, y)$ is fht the value of the filtered image pixel intensity, $I(i, j)$ is the the value of the original image in the mask pixels intensities, m and n are the filter mask width and height respectively. This algorithm is effective for slightly noisy images. In OpenCV it can be executed by using the function `cv2.blur(I, (3, 3))` for 3×3 filter kernel.

```
# Create a storage for the images filtered with arithmetic mean filter
Iblur = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
  Iblur.append((img[0], cv.blur(img[1], (3, 3))))
# Show filtered images
ShowImages(Iblur, 0)
```

Salt & Pepper (p=0.06, 1:1)

Speckle

Gaussian

Poisson

2.4 Geometric mean filter

Geometric averaging is calculated using the formula:

$$I_{filtered}(x,y)=\left[\prod_{i=0}^{m}\prod_{j=0}^{n} I(i,j)\right]^{\frac{1}{m\cdot n}}$$

The effect of applying this filter is similar to the previous method, but individual objects in the original image are less distorted. The filter can be used to suppress high-pass additive noise with better statistical performance than an arithmetic averaging filter.

This filter can be easily implemented via arithmetic mean filter since $\prod_i x_i = e^{\sum_i \ln x_i}$, so

$$I_{filtered}(x,y)=\left[\prod_{i=0}^{m}\prod_{j=0}^{n} I(i,j)\right]^{\frac{1}{m\cdot n}}=\left[e^{\sum_{i=0}^{m}\sum_{j=0}^{n}\ln I(i,j)}\right]^{\frac{1}{m\cdot n}}$$

Let's implement it for a 5×5 kernel.

```
# Define a kernel
kernel = np.ones((5, 5)) / 25
# Create a storage for the images filtered with geometric mean filter
Igeom = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
  Itmp = img[1].copy()
  if img[1].dtype == np.uint8:
    Itmp[Itmp == 0] = 1
    Ifilter =
np.power(np.exp(cv.filter2D(np.log(Itmp).astype(np.float32), -1,
kernel)), np.sum(kernel)).clip(0, 255).astype(np.uint8)
  else:
    Itmp[Itmp == 0] = sys.float_info.epsilon
    Ifilter =
np.power(np.exp(cv.filter2D(np.log(Itmp).astype(np.float32), -1,
kernel)), np.sum(kernel))
  Igeom.append((img[0], Ifilter))
# Show filtered images
ShowImages(Igeom, 0)
```

Salt & Pepper (p=0.06, 1:1)

Speckle

Gaussian

Poisson

2.5 Harmonic mean filter

The filter is based on the expression:

$$I_{filtered}(x,y)=\frac{m\cdot n}{\sum_{i=0}^{m}\sum_{j=0}^{n}\frac{1}{I(i,j)}}$$

The filter suppresses "*salt*" noise well and does not work with "*pepper*" noise.

Let's implement this filter.

At first, we need to create an image with borders, i.e., increase the image size not to think how to work with border items. As result image pixels will be shifted by the half of kernel, plus this gap will be filled with image border pixels. This is done by calling the `cv2.copyMakeBorder(I, ...)` function. It takes the following parameters: image, four gaps (top, bottom, left, right) and border type. We will use `cv2.BORDER_REPLICATE` to fill these gaps with border pixel values.

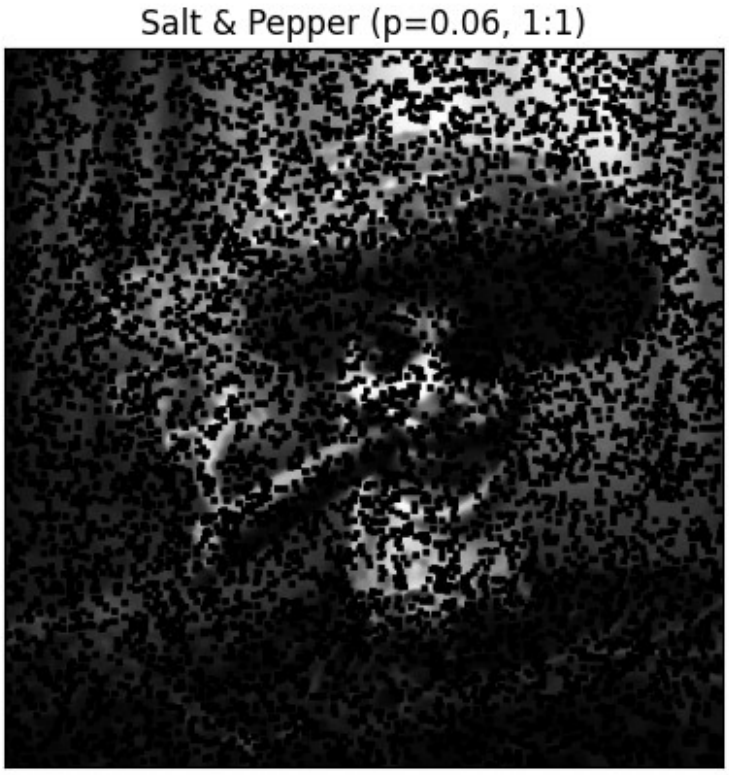
Next, we will use numpy matrix operations to calculate whole sums for the whole image at time. Don't forget that this filter works in floating point pixel values, so `uint8` images should be converted to floats and divided by 255.

```
## Execute the harmonic mean filter
## @param[in] I An image to filter
## @param[in] kernel_size The size of the filter kernel
## @return Filtered image
def HarmonicMeanFilter(I, kernel_size):
  rows, cols = I.shape[0:2]

  # Filter parameters
  kernel = np.ones((kernel_size[0], kernel_size[1]))
  #kernel = kernel / kernel_size[0] / kernel_size[1]

  # Convert to float and make image with border
  if I.dtype == np.uint8:
    Icopy = I.astype(np.float32) / 255
  else:
    Icopy = I
  Icopy[Icopy == 0] = sys.float_info.epsilon
  Icopy = cv.copyMakeBorder(Icopy,
                            (kernel_size[0] - 1) // 2, kernel_size[0]
// 2,
                            (kernel_size[1] - 1) // 2, kernel_size[1]
// 2,
                            cv.BORDER_REPLICATE)

  # Do filter
  Iout = np.zeros(I.shape, np.float32)
  for i in range(kernel_size[0]):
```

```
    for j in range(kernel_size[1]):
      Iout = Iout + kernel[i, j] / Icopy[i:i + rows, j:j + cols]
  Iout = np.sum(kernel) / Iout

  # Convert back to uint if needed
  if (I.dtype == np.uint8):
    Iout = (255 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Now we can apply it to our noisy images.

```
# Create a storage for the images filtered with a harmonic mean filter
Ihmean = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
  Ihmean.append((img[0], HarmonicMeanFilter(img[1], (5, 5))))
# Show filtered images
ShowImages(Ihmean, 0)
```



Speckle

Gaussian

It can be seen that *pepper* noise got worse, but *salt* was cleaned. Let's try filtering *salt* only noise.

```
# Add salt-type noise
Isalt = SaltAndPepper(I_jok, 0.05, 1)
# Filter image with harmonic mean filter and show the result
ShowImages([("Salt only", Isalt), ("Harmonic mean filter",
HarmonicMeanFilter(Isalt, (5, 5)))])
```

Salt only

Harmonic mean filter

2.6 Contra harmonic mean filter

The filter is based on the expression:

$$I_{filtered}(x,y)=\frac{\sum_{i=0}^{m}\sum_{j=0}^{n} I(i,j)^{Q+1}}{\sum_{i=0}^{m}\sum_{j=0}^{n} I(i,j)^{Q}}$$

where Q is the filter order. The contra harmonic filter is a generalization of the averaging filters and for $Q>0$ suppresses noises of the "*pepper*" type, and if $Q<0$ then noises of the "*salt*" type, however, it is not possible to remove white and black points at the same time. If $Q=0$ the filter turns into an arithmetic one, and if $Q=-1$ then to the harmonic one.

Implementation of the contra harmonic mean filter is very similar to a harmonic one. The only difference is that now we have to calculate top and bottom parts for the whole image separately, and then divide them.

2.6.1 Self-work

Self-work

Finish the implementation of the contra harmonic mean filter.

```
## Execute the contra harmonic mean filter
## @param[in] I An image to filter
## @param[in] kernel_size The size of the filter kernel
## @param[in] Q Filter parameter
## @return Filtered image
def ContraHarmonicMeanFilter(I, kernel_size, Q):
  # Convert to float in [0, 1] for stable exponentiation
  if I.dtype == np.uint8:
    If = I.astype(np.float32) / 255.0
  else:
    If = I.astype(np.float32)

  kernel = np.ones((kernel_size[0], kernel_size[1]), dtype=np.float32)
  kernel /= (kernel_size[0] * kernel_size[1])

  eps = 1e-12

  num = cv.filter2D(np.power(If, Q + 1), -1, kernel,
borderType=cv.BORDER_REPLICATE)
  den = cv.filter2D(np.power(If, Q), -1, kernel,
borderType=cv.BORDER_REPLICATE)

  Iout = num / (den + eps)

  # Convert back if needed
  if I.dtype == np.uint8:
    Iout = (255.0 * Iout).clip(0, 255).astype(np.uint8)
```

```
  return Iout
```

When filter implementation is finished we can apply it to our noisy images.

```
# Create a storage for the images filtered with contra harmonic mean
filter
Ichmean = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
  Ichmean.append((img[0], ContraHarmonicMeanFilter(img[1], (5, 5),
2)))
# Show filtered images
ShowImages(Ichmean, 0)
```

Speckle

Gaussian

2.6.2 Self-work

Self-work

Try different Q parameter values and tell which works better with different noise types.

```
Q_values = [-2, -1, 0, 1, 2]

for name, img in Inoise:
  res = [(f"{name} (noisy)", img)]
  for Q in Q_values:
    res.append((f"Q={Q}", ContraHarmonicMeanFilter(img, (5, 5), Q)))
  ShowImages(res, 0)
```

```
/tmp/ipykernel_292030/4068286205.py:18: RuntimeWarning: divide by zero
encountered in power
  num = cv.filter2D(np.power(If, Q + 1), -1, kernel,
borderType=cv.BORDER_REPLICATE)
/tmp/ipykernel_292030/4068286205.py:19: RuntimeWarning: divide by zero
encountered in power
  den = cv.filter2D(np.power(If, Q), -1, kernel,
borderType=cv.BORDER_REPLICATE)
/tmp/ipykernel_292030/4068286205.py:21: RuntimeWarning: invalid value
encountered in divide
  Iout = num / (den + eps)
/tmp/ipykernel_292030/4068286205.py:25: RuntimeWarning: invalid value
```

```
encountered in cast
  Iout = (255.0 * Iout).clip(0, 255).astype(np.uint8)
```

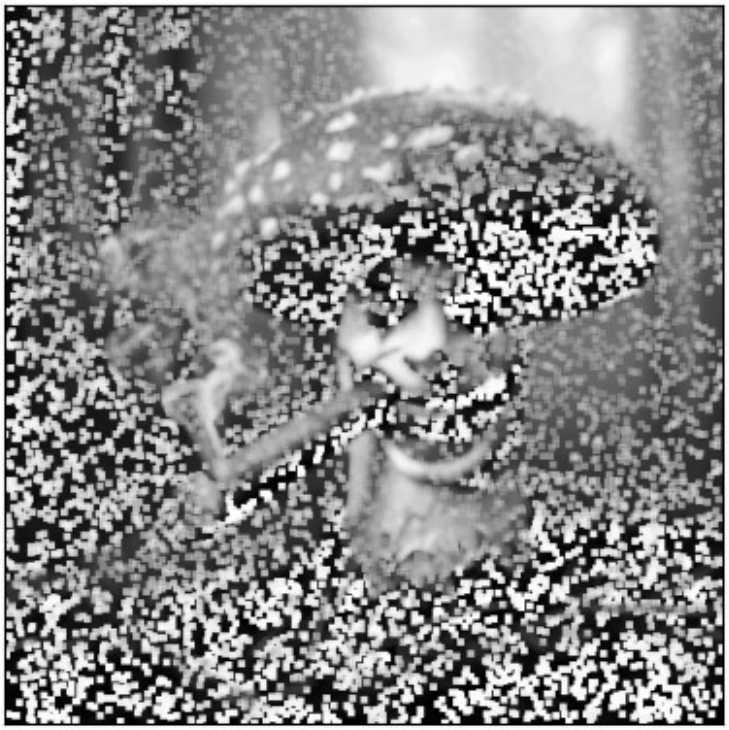
Q=-2

Q=-1

Q=0

Q=1

Q=2



Speckle (noisy)

Q=-2

Q=-1

Q=0

Q=1

Q=2

Gaussian (noisy)

Q=-2

Q=-1

Q=0

Q=1

Q=2

Poisson (noisy)

Q=-2

Q=-1

Q=0

Q=1

Q=2

ANSWER:

Salt&Pepper: Q=0

Speckle: Q=2

Gaussian: Q=1

Poisson: Q=2

2.7 Gaussian filter

The pixels in the sliding window that are closer to the analyzed pixel should have a greater influence on the filtering result than the extreme ones. Therefore, the mask weight coefficients can be described by the bell-shaped Gaussian function. During images filtering, a two-dimensional Gaussian filter is used:

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-x^2}{2\sigma^2}} \cdot \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-y^2}{2\sigma^2}}$$

The larger the parameter σ, the more the image is blurred. Typically the filter radius $r = 3\sigma$. In this case, the mask size $2r+1 \times 2r+1$ and the matrix size is $6\sigma+1 \times 6\sigma+1$. Outside this neighborhood, the values of the Gaussian function will be negligible.

OpenCV provides a function for Gaussian filter called `cv2.GaussianBlur(I, ksize, sigmaX, borderType)`. The additional parameters are used to specify the kernel size as a

tuple (e.g., `ksize = (5, 5)` for 5×5 kernel), σ value and border type (we will use `cv2.BORDER_REPLICATE`).

Let's run it.

2.7.1 Self-work

Self-work

Run the Gaussian blur filter for noisy images. Use the 5×5 kernel and $\sigma = 1.3$. Which type of noise does this filter works better with?

ANSWER: Gaussian blur works best with Gaussian-like noise (Gaussian / Speckle / Poisson).

```
Igaussblur = []

for name, img in Inoise:
  Igaussblur.append((name, cv.GaussianBlur(img, (5, 5), 1.3,
borderType=cv.BORDER_REPLICATE)))

ShowImages(Igaussblur, 0)
```

Speckle

Gaussian

Task 3. Nonlinear filtering

Process the distorted images obtained in the first point with median, weighted median, rank and Wiener filtering for different sizes of the mask and its coefficients.

Optional: Implement adaptive median filtering.

Low-pass filters are linear and are optimal when there is a normal distribution of noise in the digital image. Linear filters locally average impulse noise, smoothing images. To eliminate impulse noise, it is better to use non-linear filters, for example, *median* filters.

3.1 Median filter

The classical median filter uses a mask with unit coefficients. An arbitrary window shape can be set using zero coefficients. The pixel intensities in the window are represented as a column vector and sorted in ascending order. The filtered pixel is assigned the median (mean) intensity value in the series. The median element number after sorting can be calculated by the formula $n=\frac{N+1}{2}$, where N is the number of pixels involved in sorting. When using OpenCV, the median filter can be executed by calling `cv2.medianBlur(I, ksize)` function.

Median filter

Median filter

3.1.1 Self-work

Self-work

Run the median filter for noisy images. Which type of noise does this filter works better with?

ANSWER: Median filter works best for impulse (Salt & Pepper) noise.

```
Imedian = []

for name, img in Inoise:
  Imedian.append((name, cv.medianBlur(img, 5)))

ShowImages(Imedian, 0)
```

Speckle

Gaussian

3.2 Weighted median filter

In this median filtering modification in the mask, weights are used (numbers $2, 3$ etc.) to reflect more influence on the filtering result of pixels located closer to the element to be filtered. Each item is added the giver number of times to the array before sorting. Median filtering qualitatively removes impulse noise, and also does not introduce new intensity values in grayscale images. Increasing the size of the window increases the filter noise-canceling ability, but the objects outlines begin to distort. OpenCV does not profive weighted median filter, but it can be implemented.

```
## Weighted median filter
## @param[in] I An image to filter
## @param[in] weights Weights kernel
## @return The filtered image
def WightedMedianFilter(I, weights):
  rows, cols = I.shape[0:2]
  weights = weights.astype(np.uint16)
  kernel_size = weights.shape
  rank = int(weights.sum() // 2)

  # Convert to float and make image with border
  if I.dtype == np.uint8:
    Icopy = I.astype(np.float32) / 255
  else:
    Icopy = I
```

```
  Icopy = cv.copyMakeBorder(Icopy,
                            (kernel_size[0] - 1) // 2, kernel_size[0]
// 2,
                            (kernel_size[1] - 1) // 2, kernel_size[1]
// 2,
                            cv.BORDER_REPLICATE)

  # Fill arrays for each kernel item
  Ilayers = np.zeros(I.shape + (weights.sum(), ), dtype = np.float32)
  if I.ndim == 2:
    n = 0
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
          for k in range(weights[i, j]):
            Ilayers[:, :, n] = Icopy[i:i + rows, j:j + cols]
            n += 1
  else:
    n = 0
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
        for k in range(weights[i, j]):
          Ilayers[:, :, :, n] = Icopy[i:i + rows, j:j + cols, :]
          n += 1

  # Sort
  Ilayers.sort()

  # And select one with rank
  if I.ndim == 2:
    Iout = Ilayers[:, :, rank]
  else:
    Iout = Ilayers[:, :, :, rank]

  # Convert back to uint if needed
  if (I.dtype == np.uint8):
    Iout = (255 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Let's run this filter doubling the weight of the central point.

```
# Define weights
weights = np.ones((3, 3))
weights[1, 1] = 2
# Create a storage for the images filtered with a weighted median
filter
Iwmedian = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
```

```
  Iwmedian.append((img[0], WightedMedianFilter(img[1], weights)))
# Show filtered images
ShowImages(Iwmedian, 0)
```

Speckle

Gaussian

3.3 Rank filter

A median filtering generalization is a *rank filter* of order r which selects a pixel with the given number from the resulting column vector of mask elements $r \in [1, N)$, which will be the result of filtering.

- If the number of pixels in the window N is odd and $r = \frac{N+1}{2}$, then the rank filter is *median*.
- If $r = 1$, the filter selects the lowest intensity value and is called min*-filter*.
- If $r = N$, the filter selects the maximum intensity value and is called max*-filter*.

Sometimes rank is written as a percentage, for example, for min*-filter* rank is 0, median filter is 50, max*-filter* is 100.

It can be noted that when implementing the weighted median filter we already had the `rank` variable which was used to find the median in the varying length array. We can make it a filter parameter to make it the weighted rank filter.

```
## Rank filter
## @param[in] I An image to filter
## @param[in] weights Weights kernel
## @param[in] rank The rank to use
## @return The filtered image
def WeightedRankFilter(I, weights, rank):
```

```
  rows, cols = I.shape[0:2]
  weights = weights.astype(np.uint16)
  kernel_size = weights.shape

  # Convert to float and make image with border
  if I.dtype == np.uint8:
    Icopy = I.astype(np.float32) / 255
  else:
    Icopy = I
  Icopy = cv.copyMakeBorder(Icopy,
                            (kernel_size[0] - 1) // 2, kernel_size[0]
// 2,
                            (kernel_size[1] - 1) // 2, kernel_size[1]
// 2,
                            cv.BORDER_REPLICATE)

  # Fill arrays for each kernel item
  Ilayers = np.zeros(I.shape + (weights.sum(), ), dtype = np.float32)
  if I.ndim == 2:
    n = 0
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
          for k in range(weights[i, j]):
            Ilayers[:, :, n] = Icopy[i:i + rows, j:j + cols]
            n += 1
  else:
    n = 0
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
        for k in range(weights[i, j]):
          Ilayers[:, :, :, n] = Icopy[i:i + rows, j:j + cols, :]
          n += 1

  # Sort
  Ilayers.sort()

  # And select one with rank
  if I.ndim == 2:
    Iout = Ilayers[:, :, rank]
  else:
    Iout = Ilayers[:, :, :, rank]

  # Convert back to uint if needed
  if (I.dtype == np.uint8):
    Iout = (255 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Of course a more simple and fast modification can be used to get only a rank filter.

```
## Rank filter
## @param[in] I An image to filter
## @param[in] kernel_size The filter kernel size
## @param[in] rank The rank to use
## @return The filtered image
def RankFilter(I, kernel_size, rank):
  rows, cols = I.shape[0:2]

  # Convert to float and make image with border
  if I.dtype == np.uint8:
    Icopy = I.astype(np.float32) / 255
  else:
    Icopy = I
  Icopy = cv.copyMakeBorder(Icopy,
                            (kernel_size[0] - 1) // 2, kernel_size[0]
// 2,
                            (kernel_size[1] - 1) // 2, kernel_size[1]
// 2,
                            cv.BORDER_REPLICATE)

  # Fill arrays for each kernel item
  Ilayers = np.zeros(I.shape + (kernel_size[0] * kernel_size[1], ),
dtype = np.float32)
  if I.ndim == 2:
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
          Ilayers[:, :, i * kernel_size[1] + j] = Icopy[i:i + rows,
j:j + cols]
  else:
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
        Ilayers[:, :, :, i * kernel_size[1] + j] = Icopy[i:i + rows,
j:j + cols, :]

  # Sort
  Ilayers.sort()

  # And select one with rank
  if I.ndim == 2:
    Iout = Ilayers[:, :, rank]
  else:
    Iout = Ilayers[:, :, :, rank]

  # Convert back to uint if needed
  if (I.dtype == np.uint8):
    Iout = (255 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Let's run this filter and check the results.

3.3.1 Self-work

Self-work

Run the rank filter for min and max parameter values with all noise types.

Run the min rank filter.

```
# Create a storage for the images filtered with a min rank filter
Irank_min = []

for name, img in Inoise:
  Irank_min.append((name, RankFilter(img, (3, 3), 0))) # min in 3x3
window

ShowImages(Irank_min, 0)
```

Speckle

Gaussian

Run the max rank filter.

```
# Create a storage for the images filtered with a max rank filter
Irank_max = []

for name, img in Inoise:
  Irank_max.append((name, RankFilter(img, (3, 3), 8)))   # max in 3x3
window

ShowImages(Irank_max, 0)
```

Salt & Pepper (p=0.06, 1:1)

Speckle

Gaussian

Poisson

3.4 Adaptive median filter

In this filter modification, a sliding window of size $s \times s$ adaptively increases depending on the filtering result. Let us denote by z_{min}, z_{max}, z_{med} minimum, maximum and median values of intensities in the window, $z_{i,j}$ is the pixel intensity value with coordinates (i, j), s_{max} is the maximum allowed window size. The adaptive median filtering algorithm consists of the following steps:

1. Calculating values z_{min}, z_{max}, z_{med}, $A_1 = z_{med} - z_{min}$, $A_2 = z_{med} - z_{max}$ of the pixel (i, j) in the given window.
    a. If $A_1 > 0$ and $A_2 < 0$, go to step 2. Otherwise, increase the window size.
    b. If the current window size s is less than maximum allowed (if $s \leq s_{max}$), repeat step 1.
    c. Otherwise, the filtering result is $z_{i,j}$.
2. Calculating values $B_1 = z_{i,j} - z_{min}$, $B_2 = z_{i,j} - z_{max}$.
    a. If $B_1 > 0$ and $B_2 < 0$, the filtering result is $z_{i,j}$.
    b. Otherwise, the filtering result is z_{med}.
3. Change coordinates (i, j).
    a. If the image limit is not reached, go to step 1.
    b. Otherwise, the filtering is over.

The main idea is to increase the window size until the algorithm finds a median value that is not impulse noise, or until it reaches the maximum window size. In the latter case, the algorithm will return the value $z_{i,j}$.

3.4.1 Self-work (Optional)

> ***Self-work*** *(Optional)*
>
> *Implement the adaptive median filter and run it with different noise types. Your implementation should work both with grayscale images (image matrix has 2 dimensions) and BGR images (image matrix has 3 dimensions).*

```
## Adaptive median filter
## @param[in] I An image to filter
## @param[in] max_kernel_size The maximum filter kernel size (odd)
## @return The filtered image
def AdaptiveMedianFilter(I, max_kernel_size):
    if max_kernel_size % 2 == 0:
        max_kernel_size += 1

    # bring to (H, W, C)
    if I.ndim == 2:
        Iwork = I[..., None]
    else:
        Iwork = I

    # work in float [0,1] (or float as-is)
```

```
    If = Iwork.astype(np.float32)
    if I.dtype == np.uint8:
        If /= 255.0

    pad = max_kernel_size // 2

    # !!! IMPORTANT: np.pad preserves 3D shape even for C=1
    Ifp = np.pad(If, ((pad, pad), (pad, pad), (0, 0)), mode='edge')

    H, W, C = If.shape
    Iout = np.empty_like(If)

    for y in range(H):
        for x in range(W):
            for c in range(C):
                s = 3
                zxy = Ifp[y + pad, x + pad, c]

                while True:
                    r = s // 2
                    win = Ifp[y + pad - r:y + pad + r + 1,
                              x + pad - r:x + pad + r + 1, c]

                    zmin = win.min()
                    zmax = win.max()
                    zmed = np.median(win)

                    A1 = zmed - zmin
                    A2 = zmed - zmax

                    if (A1 > 0) and (A2 < 0):
                        B1 = zxy - zmin
                        B2 = zxy - zmax
                        if (B1 > 0) and (B2 < 0):
                            Iout[y, x, c] = zxy
                        else:
                            Iout[y, x, c] = zmed
                        break
                    else:
                        s += 2
                        if s > max_kernel_size:
                            # по методичке: вернуть z_{i,j}
                            Iout[y, x, c] = zxy
                            break

    # back to dtype
    if I.dtype == np.uint8:
        Iout = (Iout * 255.0).clip(0, 255).astype(np.uint8)
    else:
        Iout = Iout.astype(I.dtype)
```

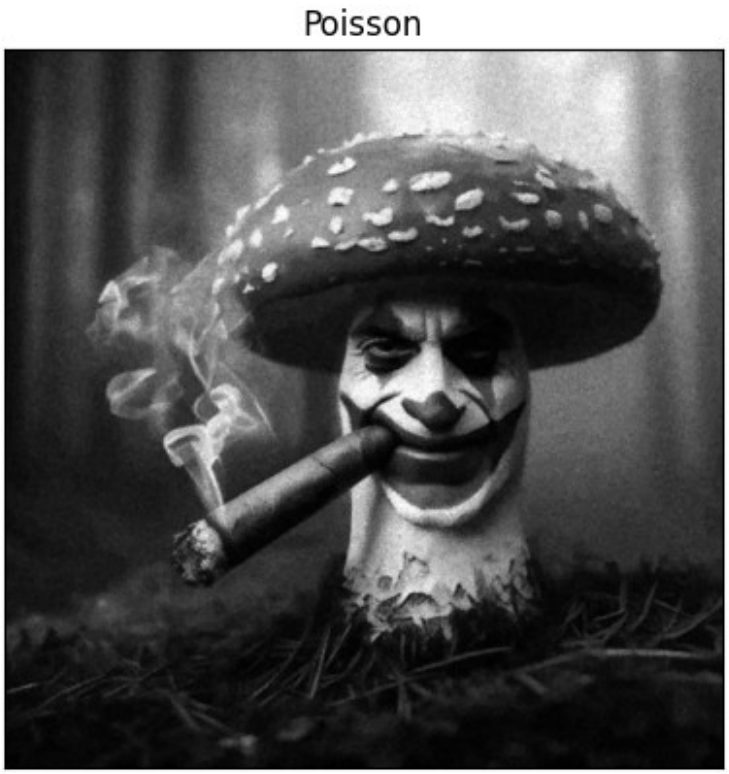
```
    return Iout[..., 0] if I.ndim == 2 else Iout
```

Now run the adaptive median filter for all noise types. Compare the results.

```
# Create a storage for the images filtered with an adaptive median
filter
Iadaptive_med = []
# Apply filter to all images and add them to an array for displaying
for img in Inoise:
  Iadaptive_med.append((img[0], AdaptiveMedianFilter(img[1], 9)))
# Show filtered images
ShowImages(Iadaptive_med, 0)
```

Speckle

Gaussian



ANSWER: Adaptive median filtering works best for salt & pepper noise: it removes impulse outliers effectively while preserving edges relatively well. For gaussian, speckle or poisson noise the improvement is worse, because the noise is not impulsive, so the filter either does not suppress it much or blurs details.

3.5 Wiener filter

Weiner filter uses Wiener's pixel-adaptive method based on statistics estimated from the local neighborhood of the each pixel.

$$\mu = \frac{1}{n \cdot m} \sum_{i=0}^{m} \sum_{j=0}^{n} I(i,j)$$

$$\sigma^2 = \frac{1}{n \cdot m} \sum_{i=0}^{m} \sum_{j=0}^{n} I^2(i,j) - \mu^2$$

$$I_{new}(x,y) = \mu + \frac{\sigma^2 - v^2}{\sigma^2} (I(x,y) - \mu)$$

Where μ is the average in the neighborhood, σ^2 is the variance, and v^2 is the noise variance.

OpenCV does not provide an implementation for this filter, but it can be easily implemented.

```
## The Weiner filter
## @param[in] img_noisy An image to filter
```

```
## @param[in] kernel Filter kernel
def WeinerFilter(I, kernel):
  rows, cols = I.shape[0:2]

  # Convert to float32
  if I.dtype == np.uint8:
    Iout = I.astype(np.float32) / 255
  else:
    Iout = np.copy(I)

  kernel_size = kernel.shape

  # Convert to float and make image with border
  if I.dtype == np.uint8:
    Icopy = I.astype(np.float32) / 255
  else:
    Icopy = I
  Icopy = cv.copyMakeBorder(Icopy,
                            (kernel_size[0] - 1) // 2, kernel_size[0]
// 2,
                            (kernel_size[1] - 1) // 2, kernel_size[1]
// 2,
                            cv.BORDER_REPLICATE)

  # Split into layers
  bgr_planes = cv.split(Icopy)
  bgr_planes_2 = []

  kernel_power = np.power(kernel, 2)

  # For all layers
  for plane in bgr_planes:
    # Calculate temporary matrices for I ** 2
    plane_power = np.power(plane, 2)

    m = np.zeros(I.shape[0:2], np.float32)
    q = np.zeros(I.shape[0:2], np.float32)

    # Calculate variance values
    for i in range(kernel_size[0]):
      for j in range(kernel_size[1]):
        m = m + kernel[i, j] * plane[i:i + rows, j:j + cols]
        q = q + kernel_power[i, j] * plane_power[i:i + rows, j:j +
cols]
    m = m / np.sum(kernel)
    q = q / np.sum(kernel)
    q = q - m * m

    # Calculate noise as an average variance
    v = np.sum(q) / I.size
```

```
    # Do filter
    plane_2 = plane[(kernel_size[0] - 1) // 2:(kernel_size[0] - 1) //
2 + rows,
      (kernel_size[1] - 1) // 2:(kernel_size[1] - 1) // 2 + cols]
    plane_2 = np.where(q < v, m, (plane_2 - m) * (1 - v / q) + m)
    bgr_planes_2.append(plane_2)

  # Merge image back
  Iout = cv.merge(bgr_planes_2)

  # Convert back to uint if needed
  if (I.dtype == np.uint8):
    Iout = (255 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

3.5.1 Self-work

Self-work

Run the Weiner filter with all of our noisy images. Use the 5×5 kernel filled with 1.

```
# Create a storage for the images filtered with a Weiner filter
Iweiner = []

kernel = np.ones((5, 5), dtype=np.float32)

for name, img in Inoise:
  Iweiner.append((name, WeinerFilter(img, kernel)))

ShowImages(Iweiner, 0)
```

```
/tmp/ipykernel_292030/3120199124.py:54: RuntimeWarning: divide by zero
encountered in divide
  plane_2 = np.where(q < v, m, (plane_2 - m) * (1 - v / q) + m)
/tmp/ipykernel_292030/3120199124.py:54: RuntimeWarning: invalid value
encountered in multiply
  plane_2 = np.where(q < v, m, (plane_2 - m) * (1 - v / q) + m)
```

Salt & Pepper (p=0.06, 1:1)

Speckle

Gaussian

Poisson

Task 4. High-pass filtering

Select source image. Detect edges with Roberts, Prewitt, Sobel, Laplace filters, Canny algorithm.

High-pass spatial filters enhance high-pass components (areas of strong intensity variation) and attenuate low-pass components of the image. They are used to highlight intensity differences and define edges (contours) in images. As a result of applying high-pass filters, the image is sharpened.

High-pass filtering

Intensity function and its first derivative, the maximum of the derivative corresponds to the edge

High-pass filters approximate the computation of directional derivatives, while the increment of the argument Δx is taken equal to 1 or 2. The main distinctive features are:

- filter mask coefficients can take negative values;
- the sum of all coefficients is zero.

4.1 Roberts filter

The Roberts filter works with the minimum dimensionality mask allowed for the derivative calculation 2×2, therefore it is fast and quite efficient. Possible options for masks for finding the gradient along the axes $O x$ and $O y$:

$$G_x = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}, G_y = \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix},$$

or

$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}.$$

As a result of applying the Roberts differential operator, we obtain an estimate of the gradient in the directions G_x and G_y. The all edge detectors gradient modulus can be calculated by the formula $G = \sqrt{G_x^2 + G_y^2} = |G_x| + |G_y|$, and gradient direction by the formula $\arctan\left(\frac{G_y}{G_x}\right)$.

In OpenCV it is implemented by calling `cv2.filter2D()` function with two matrices G_x and G_y and then calculating their square mean by `cv2.magnitude()` function.

```
## The Roberts filter
## @param[in] I An image to filter
## @return The filtered image
def RobertsFilter(I):
  # Convert to float
  if I.dtype == np.uint8:
    Iout = I.astype(np.float32) / 255
  else:
```

```
    Iout = np.copy(I)

  # Define kernels for X and Y
  kernel_x = np.array([[1, -1], [ 0, 0]])
  kernel_y = np.array([[1,  0], [-1, 0]])

  # Perform convolution
  Ix = cv.filter2D(Iout, -1, kernel_x)
  Iy = cv.filter2D(Iout, -1, kernel_y)
  Iout = cv.magnitude(Ix, Iy)

  # Convert back to uint if needed
  if I.dtype == np.uint8:
    Iout = (255 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

As this filter is not used to filter noise, so we will run it for our initial image and check what borders are found.

```
# Apply filter
Iroberts = RobertsFilter(I)
# Show the filtering result
ShowImages([("Source image", I),
            ("Roberts filter", Iroberts)], 2)
```

4.2 Prewitt filter

This approach uses two orthogonal masks of size 3×3, allowing you to more accurately calculate the derivatives along the axes $O\,x$ and $O\,y$:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

Implementation is very similar to implementation of the Roberts filter.

4.2.1 Self-work

Self-work

Implement the Prewitt filter and run it with the source image.

```
## The Prewitt filter
## @param[in] I An image to filter
## @return The filtered image
def PrewittFilter(I):
  # Convert to float
  if I.dtype == np.uint8:
    Iout = I.astype(np.float32) / 255.0
  else:
    Iout = I.astype(np.float32)

  # Define kernels for X and Y
  kernel_x = np.array([[-1, 0, 1],
                       [-1, 0, 1],
                       [-1, 0, 1]], dtype=np.float32)
  kernel_y = np.array([[-1, -1, -1],
                       [ 0,  0,  0],
                       [ 1,  1,  1]], dtype=np.float32)

  # Perform convolution
  Ix = cv.filter2D(Iout, -1, kernel_x, borderType=cv.BORDER_REPLICATE)
  Iy = cv.filter2D(Iout, -1, kernel_y, borderType=cv.BORDER_REPLICATE)
  Iout = cv.magnitude(Ix, Iy)

  # Convert back to uint if needed
  if I.dtype == np.uint8:
    Iout = (255.0 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Now run it and check results.

```
# Apply filter
Iprewitt = PrewittFilter(I)
# Show the filtering result
ShowImages([("Source image", I),
            ("Prewitt filter", Iprewitt)], 2)
```

Source image

Prewitt filter

4.3 Sobel filter

This approach is similar to the Roberts filter, but different mask weights are used. A typical example of a Sobel filter:

$$G_x=\begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}, G_y=\begin{bmatrix}1 & 2 & 1\\ 0 & 0 & 0\\ -1 & -2 & -1\end{bmatrix}.$$

This filter can be implemented manually as was described above or can use `cv2.Sobel(I, ddepth, dx, dy)` OpenCV function.

4.3.1 Self-work

Self-work

Implement the Sobel filter **without** using OpenCV `cv2.Sobel()` function. Run this filter with the source image.

```
## The Sobel filter
## @param[in] I An image to filter
## @return The filtered image
def SobelFilter(I):
  # Convert to float
  if I.dtype == np.uint8:
    Iout = I.astype(np.float32) / 255.0
  else:
    Iout = I.astype(np.float32)

  # Define kernels for X and Y
  kernel_x = np.array([[-1, 0, 1],
                       [-2, 0, 2],
                       [-1, 0, 1]], dtype=np.float32)
  kernel_y = np.array([[ 1,  2,  1],
                       [ 0,  0,  0],
```

```
                        [-1, -2, -1]], dtype=np.float32)

  # Perform convolution
  Ix = cv.filter2D(Iout, -1, kernel_x, borderType=cv.BORDER_REPLICATE)
  Iy = cv.filter2D(Iout, -1, kernel_y, borderType=cv.BORDER_REPLICATE)
  Iout = cv.magnitude(Ix, Iy)

  # Convert back to uint if needed
  if I.dtype == np.uint8:
    Iout = (255.0 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Now run it and check results.

```
# Apply filter
Isobel = SobelFilter(I)
# Show the filtering result
ShowImages([("Source image", I), ("Sobel filter", Isobel)], 2)
```

4.4 Laplace filter

Laplace filter uses approximation of the second derivatives along the axes $O\,x$ and $O\,y$ as opposed to previous approaches using the first derivative:

Laplace filter
he second derivative of the brightness function changes sign (passes through zero at the point corresponding to the edge

$$L\left(I\left(x,y\right)\right)=\frac{\partial^2 I}{\partial x^2}+\frac{\partial^2 I}{\partial y^2}$$

The above formula can be approximated by the following mask:

$$w = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Let's implement this filter according to a given forumla. It should be noted that this filter gives us the gradient, in other words, it can result in the negative values. For display purposes we will use the absolute value of the Laplace filter result (by applying `np.abs()` to the floating point value result before converting back to `uint8`). However in real cases this filter result should be used as is in floating point values for the further image processing steps.

There is the `cv2.Laplacian()` OpenCV function, however it **doesn't implement the Laplace filter**, *instead it uses the variation of the Sobel filter* along two axis with calculation of their magnitude. So it can't be used here.

4.4.1 Self-work

> **Self-work**
>
> Implement the Laplace filter **without** using OpenCV `cv2.Laplacian()` function.
> Don't forget to apply `np.abs()` to the result before converting it to $[0, 255]$ range.
> Run the Laplace filter implementation with the source image.

```
def LaplaceFilter(I):
  # Convert to float
  if I.dtype == np.uint8:
    Iout = I.astype(np.float32) / 255.0
  else:
    Iout = I.astype(np.float32)

  # Define kernel
  kernel = np.array([[ 0, -1,  0],
                     [-1,  4, -1],
                     [ 0, -1,  0]], dtype=np.float32)

  # Perform convolution (can be negative)
  Iout = cv.filter2D(Iout, -1, kernel, borderType=cv.BORDER_REPLICATE)
  Iout = np.abs(Iout)

  # Convert back to uint if needed
  if I.dtype == np.uint8:
    Iout = (255.0 * Iout).clip(0, 255).astype(np.uint8)

  return Iout
```

Now let's run it.

```
# Apply filter
Ilaplace = LaplaceFilter(I)
# Show the filtering result
ShowImages([("Source image", I), ("Laplace filter", Ilaplace)], 2)
```

4.5 Canny algorithm

One of the most widespread and efficient algorithms for extracting edges in an image is *Canny algorithm*. This algorithm allows not only to define edge pixels, but also connected boundary lines. The algorithm consists of the following steps:

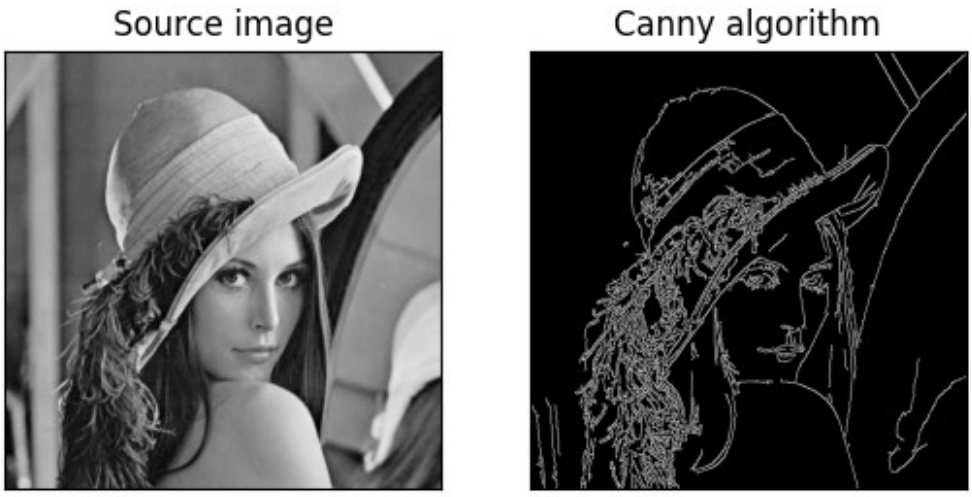
1. Elimination of small details by smoothing the original image using a Gaussian filter.
2. Using the Sobel differential operator to determine the values of the all image pixels gradient modulus, and the calculation result is rounded by steps 45°.
3. Analysis of the gradient modules values of the pixels located orthogonally to the investigated one. If the gradient modulus value of the investigated pixel is greater than orthogonal neighboring pixels, then it is *edge*, otherwise it is *non-maximum*.
4. Performing double threshold filtering of edge pixels selected in the previous step:

- If the gradient modulus value greater the threshold t_2, then the edge presence in a pixel is valid.
- If the gradient modulus value lower the threshold t_1, then the pixel is definitely not edge.
- If the gradient modulus value in a range of $\left[t_1, t_2\right)$, then such a pixel is considered as \textit {ambiguous}.

1. Suppresses all ambiguous pixels not associated with valid pixels by 8-connectivity.

In OpenCV the Canny algorithm can be executed using `cv2.Canny(I, t1, t2)` function. `t1` and `t1` parameters define thresholding step thresholds.

```
# Apply the Canny algorithm
Icanny = cv.Canny(I, 50, 220)
# Show the algorithm execution result
ShowImages([("Source image", I),
            ("Canny algorithm", Icanny)], 2)
```

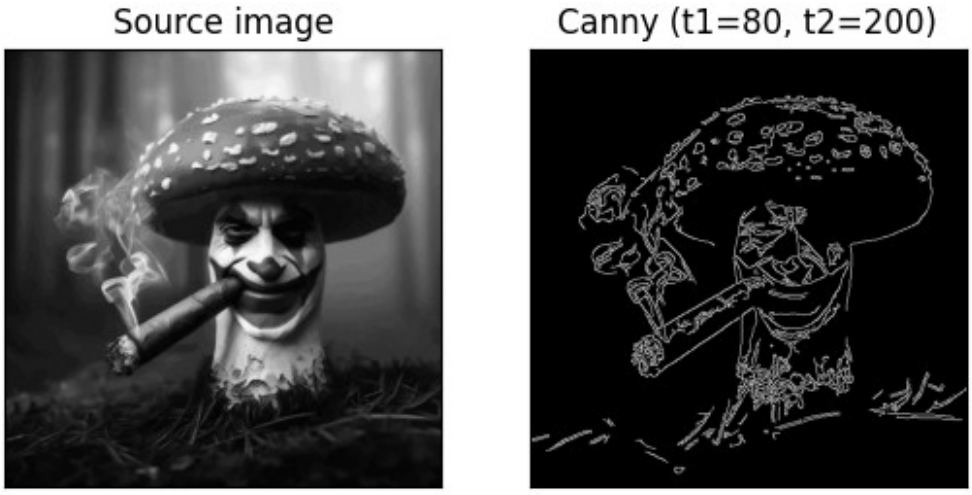


4.5.1 Self-work

Self-work

Run the Canny algorithm for your image.

```
Icanny_sw = cv.Canny(I_jok, 80, 200)

ShowImages([("Source image", I_jok), ("Canny (t1=80, t2=200)",
Icanny_sw)], 2)
```

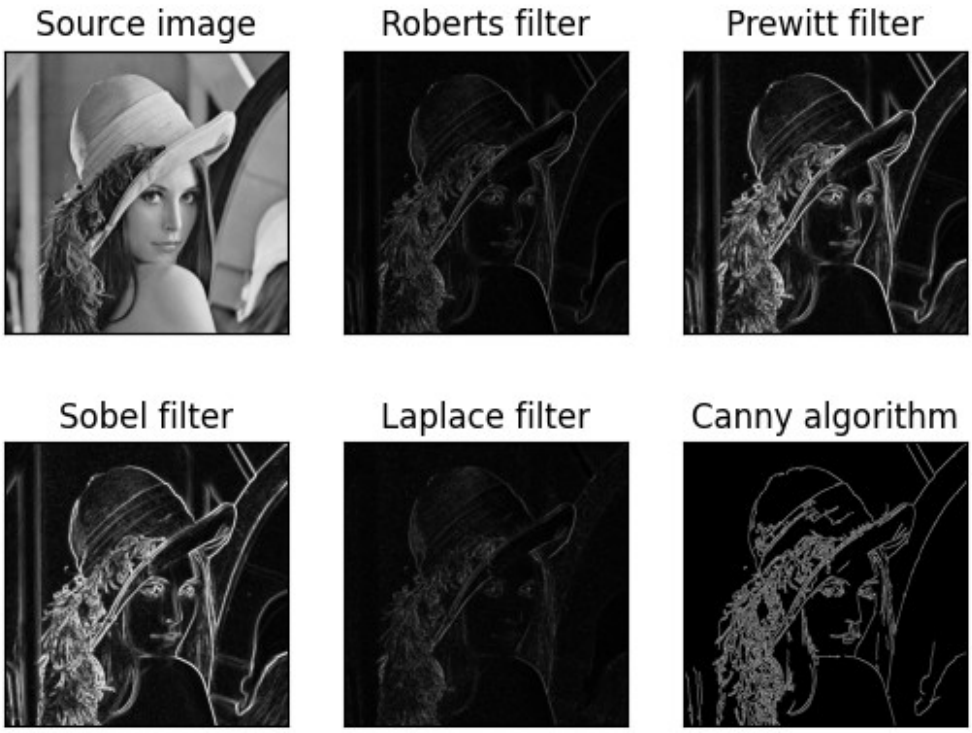


4.6 High-pass filters summary

Let's check all high-pass filter results side-by-side.

```
ShowImages([("Source image", I),
            ("Roberts filter", Iroberts),
            ("Prewitt filter", Iprewitt),
            ("Sobel filter", Isobel),
```

```
            ("Laplace filter", Ilaplace),
            ("Canny algorithm", Icanny)], 3)
```



Questions

Please answer the following questions:

- What are the main disadvantages of adaptive image filtering methods?

  Adaptive filters are computationally expensive, they depend strongly on tuning and they can over-smooth fine details or create artifacts when the noise model does not match the filter assumptions.

- Which value of the Q parameter will make the counterharmonic filter to work work as an arithmetic mean filter, and which value will make it a harmonic one?

  For the contra-harmonic mean filter: Q = 0 gives the arithmetic mean filter, and Q = -1 gives the harmonic mean filter.

- What operators can be used to detect edges in the image?

  Roberts, Prewitt, Sobel (first-derivative/gradient operators), Laplacian / LoG (second-derivative), and Canny (multi-stage detector based on smoothing + gradients + non-maximum suppression).

- Why, as a rule, is low-pass filtering performed at the first step of edge detection?

Differentiation amplifies noise, producing false edges. Low-pass smoothing suppresses noise before computing gradients/derivatives, improving edge detection stability.

Conclusion

What have you learned with this task? Don't forget to conclude it.

In this assignment we studied common noise models in digital images (salt & pepper, speckle, gaussian, and poisson) and compared the behavior of different spatial filtering methods. We generated noisy versions of a source image and applied linear low-pass filters, order-statistics filters (median, weighted median, rank min/max), the contra-harmonic mean filter with different values of the parameter (Q), and an adaptive median filter. We observed that linear smoothing filters reduce gaussian-like noise but inevitably blur edges and fine details. Median-based methods were the most effective for impulse (salt & pepper) noise, while the adaptive median filter improved robustness by increasing the window size only when needed. We also confirmed that the contra-harmonic mean filter is sensitive to (Q): for (Q>0) it suppresses pepper-like impulses and for (Q<0) it suppresses salt-like impulses. Finally, we implemented and tested several edge detection approaches (roberts, prewitt, sobel, laplace, and canny) and saw that preliminary smoothing is important to reduce false edges caused by noise. We learned how the choice of a filter should match the noise model and the desired trade-off between noise suppression and detail preservation.